

A Plain English Guide to Web Accessibility (A11y) and WCAG

By Sudeep Parchure — Angular Solutions Architect | May 2026

Because 1 in 4 people deserve to use your app — not just the ones with a mouse and perfect eyesight.

Table of Contents

1. [Why Should I Even Care?](#)
2. [Who Are We Actually Building For?](#)
3. [WCAG in Plain English — The 4 Big Rules](#)
4. [Start Here: Use the Right HTML Element](#)
5. [What is ARIA and Why Does it Exist?](#)
6. [ARIA Roles — Tell the Screen Reader What Something Is](#)
7. [ARIA Properties — Give Things Proper Names](#)
8. [ARIA States — Keep Track of What's Happening](#)
9. [Live Regions — The Secret to Dynamic Announcements](#)
10. [Building Complex Widgets the Right Way](#)
11. [Screen Readers — What Your Users Are Actually Using](#)
12. [The Mistakes Everyone Makes](#)
13. [Angular CDK — Tools That Do the Hard Work For You](#)
14. [Setting Up Your Accessibility Tools](#)
15. [Testing: How to Know if It Actually Works](#)
16. [Real Example: We Broke It, Then Fixed It](#)
17. [The 15 Things That Matter Most](#)
18. [Resources and Further Reading](#)

1. Why Should I Even Care?

Let me give you a number: **96.3%**. That is the percentage of the top one million websites that have automatically detectable accessibility failures — according to WebAIM's 2024 analysis. The average page has **56 distinct errors**.

This is not a niche problem. Over **1.3 billion people** worldwide have some form of disability. In the US alone, that's 1 in 4 adults — 61 million people. These are not rare edge cases. They are your users, your colleagues, your customers.

And here is the kicker: accessibility failures hurt *everyone*. Ever tried using a website with one hand because you were holding a coffee? Ever squinted at low-contrast text in the sun? Ever navigated a form on a phone with a broken touchscreen? Congratulations — you've experienced situational disability. Fixing accessibility fixes your product for all of those people too.

The business case (if you need one)

- The disability market controls **\$490 billion** in annual disposable income in the US alone.
- Semantic, accessible HTML is exactly what search engines reward — **SEO and a11y are the same work**.
- Between 2018 and 2023, US accessibility lawsuits exceeded **4,000 per year** — up 300%. Fixing things before shipping costs a fraction of defending a lawsuit.
- Accessible code is more modular, semantic, and testable. Teams that do this write better code overall.

The law: US (ADA, Section 508), EU (European Accessibility Act, effective 2025), UK (Equality Act 2010), Canada (AODA, Accessible Canada Act), and Australia (Disability Discrimination Act) all mandate accessibility. If you ship software to any of these markets, this is not optional.

2. Who Are We Actually Building For?

Here is a quick picture of the people who will use your app differently to how you imagine:

Who	What they deal with	What helps them
Blind users	Cannot see the screen at all	Screen readers (NVDA, JAWS, VoiceOver)
Low vision users	Partial sight, colour blindness	Magnifiers, high contrast, resizable text
Deaf/hard of hearing	Cannot hear audio	Captions, transcripts, visual alerts
Motor impairments	Tremors, one hand, paralysis	Keyboard-only, switch access, voice control
Cognitive differences	Dyslexia, ADHD, memory issues	Clear language, predictable navigation
Situational (everyone)	Broken arm, bright sun, slow phone	All of the above

The last row is the most important one to sell to stakeholders. Accessibility does not just benefit a small group with permanent disabilities — it makes your product better for everyone, in every context.

3. WCAG in Plain English — The 4 Big Rules

WCAG (Web Content Accessibility Guidelines) is the international standard for web accessibility. The current version is **WCAG 2.1**, published in 2018 by the W3C. It has **78 success criteria** organised under four principles with the acronym **POUR**.

You do not need to memorise all 78. You need to understand the four principles well enough that accessible decisions feel natural.

Perceivable — can they receive the information?

If a user cannot perceive your content, nothing else matters.

- Images need text alternatives (`alt` attributes)
- Videos need captions and transcripts
- Text must have enough contrast — **4.5:1 ratio** for normal text, 3:1 for large text
- Content can't rely on colour alone to convey meaning
- Text must be resizable to 200% without breaking the layout

Operable — can they actually use it?

If they can see your content but can't interact with it, it's still broken.

- **Everything must work with a keyboard alone** — no exceptions
- No keyboard traps (focus cannot get stuck)
- Users can skip repetitive navigation (skip links)
- No actions that require hovering or specific physical gestures

Understandable — does it make sense?

- Pages behave predictably — clicking a link doesn't unexpectedly open a new window
- Form errors are explained in text, not just highlighted in red
- The page language is declared so screen readers use the right voice

Robust — does it work across all tools?

- Valid, well-structured HTML
- Every interactive component has a name, role, and value that assistive technologies can read
- Status messages are announced without needing the user to move focus

What level should you target?

Level	What it means	Your goal
A	Bare minimum	Ship nothing below this
AA	Industry standard / legal requirement	This is your target
AAA	Enhanced / gold standard	Aim for it where you can

Level AA is what virtually every accessibility law in the world references. That is your target.

4. Start Here: Use the Right HTML Element

Before you write a single ARIA attribute, ask yourself: **is there a native HTML element that already does this?**

Native HTML elements come with roles, keyboard support, and state management built in — for free. ARIA is for when HTML alone is not enough, not a replacement for it.

The div soup disaster

This is what a screen reader hears when a page is built with `<div>` everything:

```
"My Todos Buy groceries Add a task Title Description Submit"
```

No context. No structure. No way to know what is a heading, what is an input, what is a button. The user is completely lost.

Now, with semantic HTML:

```
"My Todos, heading level 1"  
"Navigation, list, 3 items – All, Active, Completed"  
"Buy groceries, checkbox, unchecked"  
"Delete: Buy groceries, button"  
"Add a task, text field"
```

Every element is announced with its purpose. The user knows exactly where they are and what they are doing.

The semantic HTML cheat sheet

Use this element	For this purpose	What the screen reader says
<code><main></code>	Primary page content — one per page	"Main, landmark"
<code><nav></code>	Navigation link groups	"Navigation"
<code><h1></code> to <code><h6></code>	Headings and sub-headings	"Heading level 1"
<code><button></code>	Any interactive action	"Button" + works with Enter/Space
<code><a href></code>	Navigation links	"Link"
<code></code> / <code></code>	Lists of items	"List, 3 items"
<code><input type="checkbox"></code>	On/off selections	"Checkbox, unchecked"
<code><label></code>	Label for a form field	Read aloud when field receives focus
<code><div></code> / <code></code>	Layout only — no semantics	Nothing — the AT ignores it

Golden rule: If a native HTML element does what you need, use it. A `<button>` already has role, keyboard support, and disabled state. A `<div role="button">` has none of these and you have to rebuild them all yourself.

5. What is ARIA and Why Does it Exist?

Imagine you have built a custom dropdown using `<div>` and JavaScript. It looks great. It works with the mouse. But to a screen reader, it is completely invisible — just a bunch of `<div>` elements with no role, no state, no name.

ARIA (Accessible Rich Internet Applications) is a set of HTML attributes that let you add the missing semantic layer to dynamic, JavaScript-driven UIs. It tells assistive technologies three things:

- **Roles** — what an element is: `role="dialog"`, `role="tab"`, `role="combobox"`
- **Properties** — metadata that rarely changes: `aria-label`, `aria-describedby`, `aria-required`
- **States** — conditions that change at runtime: `aria-expanded`, `aria-checked`, `aria-invalid`

How the browser communicates with screen readers

Your HTML goes through several layers before a screen reader announces anything:

```
Your HTML + ARIA
↓
Browser builds the Accessibility Tree
(separate from the visual DOM)
↓
Platform Accessibility API
(Windows: UI Automation, macOS: NSAccessibility)
↓
Screen reader software (NVDA, JAWS, VoiceOver)
↓
Spoken announcement / Braille output
```

The accessibility tree is what the screen reader actually reads. It strips visual CSS, ignores decorative elements, and uses roles and ARIA to build a text model of your page. If your ARIA is wrong, the model is wrong, and the screen reader announces nonsense.

The Five Rules you must follow

1. **Use ARIA only when HTML can't do it.** A `<button>` beats `<div role="button">` every single time.
2. **Don't change what HTML already gives you.** Never override the role of a native `<button>` or `<input>`.
3. **If you add a role, you own its keyboard behaviour.** `role="button"` on a `<div>` adds no keyboard support — you must add `tabindex="0"` and handle Enter/Space yourself.
4. **Never put `aria-hidden="true"` on a focusable element.** Focus will land on it silently. The user is stranded.
5. **Every interactive element needs an accessible name.** A nameless button is announced as "button" — useless.

Three ways to hide content (and when to use each)

Method	Sighted users see it?	Screen reader reads it?	Use for
<code>aria-hidden="true"</code>	Yes	No	Decorative icons, duplicate text
<code>display:none</code>	No	No	Hidden panels, modals before open
<code>.sr-only</code> CSS class	No	Yes	Labels you want AT to hear but not show

The `.sr-only` class is your best friend for adding context that sighted users don't need to see. Tailwind includes it out of the box.

6. ARIA Roles — Tell the Screen Reader What Something Is

A role is the single most important piece of information in the accessibility tree. It tells the screen reader what kind of thing an element is — before the name, before the state, before anything else.

WAI-ARIA 1.2 defines **82 roles** in four main categories:

Landmark roles — the page's table of contents

Screen reader users press a single key (`H` in NVDA) to jump between landmarks — like a table of contents for the page structure. Get these wrong and navigating your page is like reading a book with no chapters.

Role	HTML equivalent	Key rule
<code>main</code>	<code><main></code>	One per page only. Two <code><main></code> landmarks breaks navigation.
<code>navigation</code>	<code><nav></code>	Add <code>aria-label</code> when you have more than one nav
<code>banner</code>	<code><header></code>	One per page — the page-level header
<code>contentinfo</code>	<code><footer></code>	One per page — the page-level footer
<code>region</code>	<code><section></code>	Needs <code>aria-label</code> or <code>aria-labelledby</code> to be visible as a landmark
<code>search</code>	<code><search></code>	Search forms

What goes wrong: If your Angular component adds `role="main"` to its host element while `app.html` already has `<main>`, you now have **two main landmarks**. NVDA users pressing `D` see "Main" listed twice in the landmarks menu. They have no idea which one is the real main content. The fix: use `role="region"` for sub-sections, never `role="main"` on components.

Widget roles — interactive controls

When you use a widget role, you enter a contract: implement the expected keyboard behaviour and maintain the expected states. Deviating from the contract breaks users' muscle memory.

Role	What it is	What keyboard expects
<code>button</code>	Clickable action — just use <code><button></code>	Enter or Space to activate
<code>checkbox</code>	Toggle — supports true/false/mixed	Space to toggle
<code>dialog</code>	Modal overlay	Escape to close, Tab stays inside
<code>tab + tabpanel</code>	Tabbed interface	Arrow keys to switch tabs
<code>combobox</code>	Text input with suggestions	Arrow Down to open, Enter to select
<code>switch</code>	On/off toggle	Space to toggle — never use <code>mixed</code>
<code>slider</code>	Range input	Arrow keys to change value

What goes wrong: You use `role="button"` on a `<div>`. It looks like a button. Mouse users click it — it works. But keyboard users press Tab to reach it... and nothing happens. They press Enter... nothing. Space... nothing. You have the label of a button with none of the behaviour. The fix: use `<button>`. If you truly cannot, add `tabindex="0"` and handle both `keydown` Enter and Space events.

7. ARIA Properties — Give Things Proper Names

Think of properties as the metadata of an element. They answer: what is this thing called? What does it describe? How does it relate to other elements?

How screen readers decide what to announce as a name

The browser uses a priority order to figure out the accessible name of an element. The first source it finds wins:

- `aria-labelledby` — points to another element's text (highest priority)
- `aria-label` — an inline string you provide
- `<label>` element — the native way for form inputs
- The element's own text content — for buttons and links
- `title` attribute — last resort, unreliable across screen readers

```

<!-- Best for form inputs: native label -->
<label for="task-title">Task title</label>
<input id="task-title" type="text" />
<!-- Screen reader: "Task title, text field" -->

<!-- Best for icon buttons: aria-label -->
<button aria-label="Delete: Buy groceries">
  <svg aria-hidden="true"><!-- icon --></svg>
</button>
<!-- Screen reader: "Delete: Buy groceries, button" -->

<!-- Best for sections: aria-labelledby points to visible heading -->
<h2 id="active-section">Active Tasks</h2>
<section aria-labelledby="active-section">...</section>
<!-- Screen reader: "Active Tasks, region" -->

```

What goes wrong — icon button with no name: You have a trash icon button with no label. Every screen reader user hears "button, button, button" repeated for every list item. They have absolutely no idea which item they're about to delete. The fix: `aria-label="Delete: Buy groceries"` and `aria-hidden="true"` on the SVG. Now they hear "Delete: Buy groceries, button".

Describing vs labelling

- `aria-label` / `aria-labelledby` = the **name** — short, announced first
- `aria-describedby` = the **description** — longer, announced after the name

```

<input id="task-title" aria-required="true" aria-invalid="true" aria-describedby="title-error" />
<p id="title-error" role="alert">Title cannot be empty</p>

```

The screen reader announces: *"Task title, text field, required, invalid — Title cannot be empty"*

Relationship properties you should know

Property	What it tells the screen reader
<code>aria-describedby</code>	"This description gives more context about me"
<code>aria-controls</code>	"I control that other element over there"
<code>aria-haspopup</code>	"Activating me opens a menu / listbox / dialog"
<code>aria-required</code>	"This field must be filled before submitting"

8. ARIA States — Keep Track of What's Happening

Properties are set once and barely change. States change constantly as the user interacts — expanded/collapsed, checked/unchecked, valid/invalid. Every state change updates the accessibility tree, and the screen reader picks it up in real time.

The most important states

`aria-expanded` — for menus, accordions, dropdowns

```

<button [attr.aria-expanded]="isOpen()" aria-controls="filter-panel">Filter options</button>
<div id="filter-panel" [hidden]="!isOpen()">...</div>

```

- Screen reader when closed: *"Filter options, button, collapsed"*
- Screen reader when open: *"Filter options, button, expanded"*

What goes wrong: You have a dropdown that opens visually but there's no `aria-expanded`. The screen reader announces "Filter options, button" every time — the user has no idea if it opened or not. They click it twice trying to get a response.

`aria-invalid` — for form validation

```
<input [attr.aria-invalid]="hasError() ? 'true' : null" />
```

Use `null` to remove the attribute — not `'false'`. `aria-invalid="false"` is technically valid but adds "invalid: false" noise to every announcement.

What goes wrong: You show a red border when a field fails validation, but you don't set `aria-invalid`. Sighted users see the red. Screen reader users hear nothing different. They submit the form again and again wondering why it's not working.

`aria-disabled` vs native `disabled`

These are very different and the distinction matters:

	Native <code>disabled</code>	<code>aria-disabled="true"</code>
Keyboard reachable?	No — removed from tab order	Yes — still focusable
AT announces it?	No — invisible to screen readers	Yes — "unavailable" / "dimmed"
Mouse clickable?	No	No (you prevent in JS)

When to use `aria-disabled`: When you want keyboard users to *find* the button and understand why it is unavailable — like a Submit button that is waiting for form completion. With native `disabled`, they cannot even find it.

`aria-checked` — for checkboxes and toggle patterns

Supports three values: `'true'`, `'false'`, and `'mixed'` (for "select all" with partial selection).

`aria-current` — mark where the user currently is

```
<!-- In a navigation bar -->
<a [attr.aria-current]="isCurrentPage ? 'page' : null">Dashboard</a>
<!-- Screen reader: "Dashboard, link, current page" -->
```

`aria-busy` — during loading

```
<section [attr.aria-busy]="isLoading()">...</section>
<!-- Screen reader: "Todo list, region, busy" -->
```

9. Live Regions — The Secret to Dynamic Announcements

Here is a scenario. A user submits a form. Your app saves the data and shows a green "Saved!" toast in the corner.

A sighted user sees it immediately.

A screen reader user? **They hear nothing.** The focus didn't move. The screen reader wasn't looking at that corner. The save happened in complete silence. The user has no idea if it worked.

This is the problem live regions solve.

What is a live region?

A live region is an element you mark as "watch this area — if the content changes, announce it automatically, even without focus moving there."

```
<!-- This div always exists in the DOM (even when empty) -->
<div role="status" aria-live="polite" class="sr-only">
  @if (statusMessage()) { {{ statusMessage() }} }
</div>
```

When the content inside this div changes, the screen reader announces it at the next opportunity.

Polite vs assertive — know the difference

Setting	When screen reader announces	Use it for
aria-live="polite"	After the user finishes what they are saying	Saves, adds, filter changes, success messages
aria-live="assertive"	Immediately — interrupts everything	Deletes, critical errors, session timeouts only

What goes wrong: You use `aria-live="assertive"` for every notification. Every small thing — adding a task, filtering results, closing a menu — interrupts the user mid-sentence. They get increasingly frustrated and lose their place. Rule of thumb: if it's not a deletion or a critical error, use `polite`.

The timing trap — the most common live region mistake

The live region element must exist in the DOM before content is injected into it.

If you create the region and fill it at the same time (in the same render cycle), most screen readers silently ignore it. The region must already be present — even if empty.

```
<!-- WRONG: the region is created at the same time as the content -->
@if (message()) {
  <div role="status">{{ message() }}</div>
}

<!-- RIGHT: region always in DOM, content changes inside it -->
<div role="status" class="sr-only">@if (message()) { {{ message() }} }</div>
```

Shortcut roles that bundle everything together

Role	Is the same as	When to use
role="alert"	assertive live + atomic	Errors, urgent messages
role="status"	polite live + atomic	Success messages, info
role="log"	polite live	Chat feeds, activity logs

In Angular, use LiveAnnouncer

Angular CDK's `LiveAnnouncer` handles the live region internally and works around browser quirks you don't want to debug:


```
import { LiveAnnouncer } from '@angular/cdk/ally';

private readonly liveAnnouncer = inject(LiveAnnouncer);

onDeleteTodo(title: string): void {
  this.todoService.delete(id);
  // Assertive — this is destructive, user must know immediately
  this.liveAnnouncer.announce(`${title}" deleted`, 'assertive');
}

onFilterChanged(filter: string): void {
  const count = this.filteredTodos().length;
  // Polite — wait for user to finish reading
  this.liveAnnouncer.announce(
    `Showing ${count} ${filter} task${count !== 1 ? 's' : ''}`,
    'polite'
  );
}
```

SPAs need to announce route changes

Angular's router does not automatically tell screen readers when the page changes. A user presses a link, the URL changes, the page content changes — but the screen reader hears nothing because no focus moved and no live region updated.

Fix: a service that listens for `NavigationEnd` and announces the new page title:

```
router.events.pipe(filter((e) => e instanceof NavigationEnd)).subscribe(() => {
  liveAnnouncer.announce(titleService.getTitle(), 'polite');
});
```

10. Building Complex Widgets the Right Way

When you need to build something beyond a basic button or input — a tab panel, a modal, a dropdown list — there is an official guide that tells you exactly how to do it.

The **ARIA Authoring Practices Guide (APG)** at w3.org/WAI/ARIA/apg/patterns defines the complete contract for every complex widget: the roles needed, the keyboard interactions required, and how focus should move. Always check the APG before you start building.

Modal dialog — the one everyone gets wrong

A modal dialog that doesn't trap focus is a WCAG failure. A screen reader user opens it and presses Tab — focus escapes the dialog and lands on content behind it. They are interacting with the page as if the dialog doesn't exist.

The correct setup:

```
<div
  role="dialog"
  aria-modal="true"
  aria-labelledby="dialog-title"
  cdkTrapFocus
  cdkTrapFocusAutoCapture
>
  <h2 id="dialog-title">Confirm Delete</h2>
  <p>Are you sure you want to delete this task?</p>
  <button (click)="confirm()">Delete</button>
  <button (click)="close()">Cancel</button>
</div>
```

- `role="dialog"` — tells the AT it's a dialog
- `aria-modal="true"` — tells the AT to ignore background content
- `aria-labelledby` — links the dialog to its title
- `cdkTrapFocus` — keeps Tab inside the dialog
- On close: **return focus to the button that opened it**

Tab panels — the arrow key contract

```
role="tablist"
├ role="tab" aria-selected="true"  aria-controls="panel-all"
├ role="tab" aria-selected="false" aria-controls="panel-active"
└ role="tab" aria-selected="false" aria-controls="panel-done"

role="tabpanel" id="panel-all" aria-labelledby="tab-all"
```

Keyboard contract:

- `Tab` — moves into the tablist, lands on the active tab only
- `Arrow Left/Right` — switches between tabs
- `Home / End` — first/last tab
- `Tab` from the active tab — jumps into the tab panel content

What goes wrong: You have three buttons for All / Active / Completed filters, with no tab semantics. The screen reader announces three plain buttons — no indication they're related, no indication which one is selected. A keyboard user can `Tab` through all three but pressing `Left/Right` does nothing. The fix: `role="tablist"` on the container, `role="tab"` on each button, `[attr.aria-selected]="isActive"` .

Angular CDK — use it before building custom

What you need	Use this CDK tool	Why
Focus trap in a modal	<code>cdkTrapFocus</code> directive	One attribute, fully handled
Arrow key navigation in a list	<code>ActiveDescendantKeyManager</code>	Handles wrap, Home/End, typeahead
Detect keyboard vs mouse focus	<code>FocusMonitor</code>	Shows focus rings only for keyboard
SPA route announcements	<code>LiveAnnouncer</code>	One method call, no bugs

11. Screen Readers — What Your Users Are Actually Using

You cannot test accessibility without understanding what your users are running. Here is the real picture from WebAIM's 2024 survey:

Screen reader	Share	Platform	Cost
JAWS	~46%	Windows	Paid (enterprise)
NVDA	~30%	Windows	Free and open source
VoiceOver (iOS)	~11%	iPhone / iPad	Built-in, free
VoiceOver (macOS)	~9%	Mac	Built-in, free
TalkBack	~7%	Android	Built-in, free
Narrator	~2%	Windows 11	Built-in, free

The minimum test set

Test with these three combinations to cover roughly 85% of real users:

Screen reader	Browser	Why
NVDA	Firefox	Free, most common pairing among AT users
JAWS	Chrome or Edge	Dominant in enterprise environments
VoiceOver	Safari	Only reliable Apple pairing

Why different screen readers behave differently

Screen readers don't simply "read" your DOM. They build a **virtual buffer** — a text snapshot of the page — and let users navigate that with their own keyboard shortcuts that have nothing to do with browser focus.

This means:

- Dynamically added content may not appear until the buffer refreshes
- Your visual focus outline and the screen reader's reading cursor are separate things
- A feature that works in NVDA+Firefox may behave differently in JAWS+Chrome

There is no substitute for testing with the actual software.

How the same ARIA looks across screen readers

Attribute	NVDA + Firefox	JAWS + Chrome	VoiceOver macOS
<code>aria-expanded="false"</code>	"collapsed"	"collapsed"	"collapsed"
<code>aria-expanded="true"</code>	"expanded"	"expanded"	"expanded"
<code>aria-invalid="true"</code>	"invalid entry"	"invalid entry"	"invalid"
<code>role="alert"</code> fires	Interrupts immediately	Interrupts immediately	Interrupts immediately
<code>aria-live="polite"</code> fires	After current speech	After current speech	After current speech

12. The Mistakes Everyone Makes

These are the most common accessibility bugs, what the user experiences, and how to fix them.

Focus management mistakes

Ghost focus — focusing an element with no name

```
<!-- WRONG — focus moves but screen reader says nothing -->
<div tabindex="-1" id="modal">...</div>

<!-- RIGHT — role + label = proper announcement -->
<div role="dialog" aria-labelledby="modal-title" tabindex="-1">
  <h2 id="modal-title">Confirm Delete</h2>
</div>
```

Screen reader experience (wrong): User opens modal → complete silence → they have no idea a dialog appeared.

Not returning focus when a dialog closes

You close a modal and focus drops to `document.body`. The user is somewhere random on the page with no orientation. Always return focus to **the button that triggered the dialog**.

@if removing the focused element

In Angular, when an `@if` block removes the currently-focused element, focus falls to `document.body`. Before removing an element, check if it has focus and move it explicitly.

Role mistakes

`role="button"` on a `<div>` when you could use `<button>`

A `<div role="button">` has no keyboard support out of the box. Users pressing Tab may not even reach it. Those who do press Enter or Space — and nothing happens. Use `<button>`. Always.

Two `<main>` landmarks

An Angular component with `host: { 'role': 'main' }` inside a shell that already has `<main>`. Screen readers list two "Main" landmarks. Users are confused about which is which. Fix: components use `role="region"` with an `aria-label`.

`role="region"` with no accessible name

A region without `aria-label` or `aria-labelledby` is not listed as a landmark at all in most screen readers. The whole point of adding the role is lost.

Label mistakes

Icon-only button with no name

```
<!-- WRONG -- announces "button" 10 times for 10 items -->
<button type="button"><svg>...</svg></button>

<!-- RIGHT -- announces "Delete: Buy groceries, button" -->
<button type="button" [attr.aria-label]="Delete: ' + todo().title">
  <svg aria-hidden="true">...</svg>
</button>
```

Screen reader experience (wrong): User navigates to 10 delete buttons and hears "button, button, button, button...". They cannot tell which item each button acts on.

Redundant labels — "Submit button, button"

`aria-label="Submit button"` on a `<button>`. The role announces "button" automatically — you end up with "Submit button, button". Just use `aria-label="Submit"`.

`aria-labelledby` pointing to a non-existent ID

The screen reader finds no matching element, announces the role only, and the user hears nothing useful. Always double-check your ID references.

Live region mistakes

Creating the region and its content in the same render cycle

Screen readers silently ignore regions that didn't exist before their content appeared. Keep the region in the DOM always, empty, and inject content into it.

`aria-live="assertive"` for everything

Assertive interrupts the user mid-word. Reserve it for destructive actions and critical errors. Use `polite` for everything else.

Keyboard mistakes

Mouse-only interactions

Hover menus with no keyboard equivalent. Drag-and-drop with no keyboard alternative. Right-click menus with no other access path. If it requires the mouse, it fails WCAG 2.1.1.

Keyboard traps

Focus enters a component and can't leave without a mouse click. This is a WCAG 2.1.2 (Level A) failure. The user is completely stuck.

Forgetting `event.preventDefault()` on arrow keys

Arrow key handlers in custom menus and grids must call `event.preventDefault()` to stop the browser from scrolling the page while the user is navigating your widget.

Angular-specific pitfalls

OnPush + RxJS-derived ARIA bindings not updating

Signal-based ARIA bindings (`[attr.aria-label]="signal()"`) automatically trigger change detection in OnPush components. RxJS-derived values need explicit `markForCheck()` calls.

Dynamic IDs breaking SSR hydration

IDs generated with `Math.random()` differ between the server render and client hydration. The `aria-labelledby` reference breaks. Use stable deterministic IDs.

`@defer` blocks exposing incomplete content to AT

Lazily loaded blocks can appear partially rendered in the accessibility tree. Use `[attr.aria-busy]="true"` on the parent region while deferred content loads.

13. Angular CDK — Tools That Do the Hard Work For You

Angular's CDK (Component Dev Kit) includes a complete accessibility module that handles the complicated parts — browser quirks, SSR safety, cleanup — so you don't have to.

Install it once, use it everywhere

```
npm install @angular/cdk
```

No module registration. All CDK a11y services are `providedIn: 'root'`.

LiveAnnouncer — programmatic screen reader announcements

The safest way to make announcements. CDK manages the live region element internally.

```
private readonly liveAnnouncer = inject(LiveAnnouncer);

// When a task is added
this.liveAnnouncer.announce(`${title}" added`, 'polite');

// When a task is deleted
this.liveAnnouncer.announce(`${title}" deleted`, 'assertive');

// When a filter changes
this.liveAnnouncer.announce(
  `Showing ${count} ${filter} task${count !== 1 ? 's' : ''}`,
  'polite'
);
```

FocusTrap — keep focus inside dialogs

```
// In your dialog component constructor:
afterNextRender(() => {
  this.focusTrap = this.focusTrapFactory.create(this.elementRef.nativeElement);
  this.focusTrap.focusInitialElementWhenReady();
});

ngOnDestroy(): void {
  this.focusTrap?.destroy();
}
```

Or with zero code, just the directive:

```
<div role="dialog" cdkTrapFocus cdkTrapFocusAutoCapture>
  <!-- Tab key cannot escape this element -->
</div>
```

FocusMonitor — know if the user is on keyboard or mouse

```
private readonly focusMonitor = inject(FocusMonitor);
protected readonly focusOrigin = signal<FocusOrigin | null>(null);

constructor() {
  afterNextRender(() => {
    this.focusMonitor.monitor(this.elementRef).subscribe(origin => {
      this.focusOrigin.set(origin); // 'keyboard' | 'mouse' | 'touch' | null
    });
  });
}
```

Use this to show focus rings only for keyboard users — the WCAG 2.4.7 pattern.

Signal-based ARIA in Angular 21 — the right pattern

```
// The label updates automatically when filter changes
protected readonly ariaListLabel = computed(() => ({
  all: 'All tasks',
  active: 'Active tasks',
  completed: 'Completed tasks',
})[this.filter()]);

// Null removes the attribute — correct for valid fields
protected readonly titleAriaInvalid = computed(() =>
  this.formErrors().title ? 'true' : null
);
```

```
<ul [attr.aria-label]="ariaListLabel()">
  ...
</ul>
<input [attr.aria-invalid]="titleAriaInvalid()" />
```

Host bindings — put ARIA directly on the component element

```
@Component({
  selector: 'app-todo-list',
  host: {
    role: 'region',
    '[attr.aria-label]': 'ariaLabel()',
  },
})
export class TodoListComponent {
  protected readonly ariaLabel = computed(
    () => `Todo list — ${this.stats().total} items, ${this.stats().active} active`,
  );
}
```

This puts the role and label on `<app-todo-list>` itself — no redundant wrapper div needed.

14. Setting Up Your Accessibility Tools

Everything listed here is **free**. You can have a working accessibility testing setup in under 30 minutes.

Angular CDK

```
npm install @angular/cdk

# Verify it installed correctly:
npm list @angular/cdk
```

axe-core — automated testing in your unit tests

```
# For Vitest (Angular 21 default):
npm install --save-dev axe-core vitest-axe
```

Add this to any component spec and it runs a full WCAG audit automatically:

```
import { axe, toHaveNoViolations } from 'vitest-axe';
expect.extend(toHaveNoViolations);

it('passes axe accessibility checks', async () => {
  const results = await axe(fixture.nativeElement);
  expect(results).toHaveNoViolations();
});
```

NVDA — free screen reader for Windows

1. Download at nvaccess.org/download
2. Install (no reboot needed)
3. Launch with **Ctrl+Alt+N**
4. Pair with **Firefox** for most reliable results

Key	What it does
NVDA+F7	List all page landmarks
NVDA+F6	List all headings
D	Jump to next landmark
H	Jump to next heading
B	Jump to next button
F	Jump to next form field
NVDA+Space	Switch between browse and forms mode

Chrome browser extensions (all free)

Tool	How to get it	Best for
axe DevTools	Chrome Web Store → search "axe devtools"	Detailed WCAG rule breakdown
WAVE	Chrome Web Store → search "WAVE"	Visual error overlay on page
Colour Contrast Analyser	TPGi.com → free download	Checking exact contrast ratios

15. Testing: How to Know if It Actually Works

No single tool catches everything. Use all four layers together.

The four-layer approach

Layer	Tools	Catches
Automated unit tests	axe-core, vitest-axe	~35% of WCAG issues
Automated page audit	Lighthouse, axe DevTools, WAVE	~35% (some overlap)
Keyboard-only testing	Just your browser	Focus traps, tab order, missing keyboard support
Screen reader testing	NVDA + Firefox minimum	Announcements, live regions, AT-specific bugs

Automated tools combined catch about 40% of real issues. The other 60% requires a human. Both are essential.

Quick: run Lighthouse right now

1. Open Chrome → your app
2. **F12** → Lighthouse tab
3. Tick "Accessibility" only → Analyse page load
4. Target score: **90+**

Lighthouse automatically checks: colour contrast, form labels, alt text, valid ARIA, landmark presence, skip link, heading order.

Keyboard test — the fastest manual check

Unplug your mouse (or just don't touch it). Open your app. Tab through everything.

Ask yourself:

- Can I see a visible focus ring on every element I Tab to?
- Does the Tab order make sense left-to-right, top-to-bottom?
- Does the skip link appear on my very first Tab press?
- Can I open menus, submit forms, delete items, and change filters — all without touching the mouse?
- Does Escape close every dialog and menu?
- Is there anywhere focus gets stuck?

If you answer "no" to any of these, you have a WCAG failure.

Screen reader test with NVDA — what to check

1. **NVDA+F7** → every expected landmark should appear (main, navigation, footer, regions)
2. **NVDA+F6** → heading hierarchy should make sense (H1 → H2 → H3, no skips)
3. Tab through buttons → each should announce its name and role
4. Submit the form → success or error should be announced
5. Delete an item → deletion announcement should interrupt immediately
6. Change a filter → new count should be announced politely

Test ARIA attribute updates in your specs

```
it('sets aria-invalid when the field has an error', () => {
  component.formErrors.set({ title: 'Required', description: '' });
  fixture.detectChanges();
  const input = fixture.nativeElement.querySelector('#task-title');
  expect(input.getAttribute('aria-invalid')).toBe('true');
});

it('removes aria-invalid when the error is cleared', () => {
  component.formErrors.set({ title: '', description: '' });
  fixture.detectChanges();
  const input = fixture.nativeElement.querySelector('#task-title');
  expect(input.getAttribute('aria-invalid')).toBeNull(); // null, not 'false'
});

it('announces deletion assertively', () => {
  const spy = vi.spyOn(liveAnnouncer, 'announce');
  component.onDelete('todo-1');
  expect(spy).toHaveBeenCalledWith(expect.stringContaining('deleted'), 'assertive');
});
```

16. Real Example: We Broke It, Then Fixed It

Here is a real accessibility audit on an Angular 21 todo application. Twelve issues. All found, all fixed.

What we started with

Severity	Issues found	Result
Critical (WCAG Level A failure)	4	All fixed
Serious (WCAG Level AA failure)	4	All fixed
Moderate (best practice)	4	All fixed

Lighthouse before: 72 / 100 — after: 98 / 100

axe before: 7 Critical + 4 Serious — after: 0 Critical, 0 Serious

Critical issue 1: Two `<main>` landmarks

What was wrong: `TodoListComponent` had `host: { 'role': 'main' }`. The shell already had `<main>`. Two main landmarks.

What screen readers experienced: NVDA's landmark list showed "Main" twice. Users had no idea which was the real main content.

The fix: Changed `role: 'main'` → `role: 'region'` with an `aria-label` computed from live stats.

Critical issue 2: No skip link

What was wrong: No "Skip to main content" link.

What keyboard users experienced: Every page load required Tabbing through the entire header navigation before reaching any todo content. For a user navigating with a switch or keyboard, this is exhausting — every. single. time.

The fix:

```
<a href="#main-content" class="sr-only focus:not-sr-only focus:absolute focus:top-4 focus:left-4">
  Skip to main content
</a>
<main id="main-content" tabindex="-1">...</main>
```

The `tabindex="-1"` on `<main>` is essential — without it, Safari doesn't focus the element correctly when the skip link is clicked.

Critical issue 3: `<nav>` with no name

What was wrong: Plain `<nav>` with no `aria-label`.

What screen readers experienced: NVDA announced "navigation" — no context about which navigation. If there were two navs, users couldn't tell them apart.

The fix: Added `aria-label="Primary"`. Now: *"Primary, navigation, landmark"*

Critical issue 4: Colour contrast failure

What was wrong: Amber-400 background (`#f59e0b`) with white text. Contrast ratio: **2.1:1**.

WCAG requires: 4.5:1 for normal text.

The fix: Changed to stone-900 (`#1c1917`) text on the same background. Contrast: **8.6:1**.

Serious issue: Icon-only buttons

What was wrong: Every Edit and Delete button had only an SVG icon. No label.

What screen readers experienced: Users navigating by buttons heard "button, button, button, button..." — 10 identical announcements for 10 items. No way to know which item each button was for.

The fix:

```
<button [attr.aria-label]='Delete: ' + todo().title">
  <svg aria-hidden="true">...</svg>
</button>
```

Now: *"Delete: Buy groceries, button"*. Crystal clear.

Serious issue: No live announcements

What was wrong: Adding, deleting, toggling, filtering — all in silence.

What screen readers experienced: User adds a task → nothing announced. User deletes a task → nothing announced. User has no confirmation that anything happened.

The fix: Installed `@angular/cdk` and added `LiveAnnouncer` to all five actions:

```
Add:      "Task 'Buy milk' added"      → polite
Delete:    "'Buy milk' deleted"         → assertive
Toggle:    "'Buy milk' marked complete" → polite
Filter:    "Showing 3 active tasks"     → polite
Clear:     "2 completed tasks cleared"  → polite
```

17. The 15 Things That Matter Most

If you take nothing else from this guide, take these:

1. **Use semantic HTML first.** Every correct native element is an ARIA attribute you don't have to write.
2. **One `<main>` per page.** Components use `role="region"`, never `role="main"`.
3. **Skip links are non-negotiable.** WCAG Level A. Every page. Always.
4. **Every icon button needs an aria-label.** `aria-label="Delete: Buy groceries"`. Not just "Delete".
5. **Set `aria-invalid`, not just a red border.** Sighted users see colour. Screen reader users need the attribute.
6. **Keep live regions in the DOM always.** Empty is fine. Appearing at the same time as content breaks announcements.
7. **Use `polite` by default. `assertive` for deletions and critical errors only.**
8. **SPAs need route announcements.** Angular doesn't do this automatically.
9. **Install `@angular/cdk`.** `LiveAnnouncer`, `FocusTrap`, `FocusMonitor` — don't rebuild what CDK gives you.
10. **Consult the APG before building any custom widget.** The keyboard contract is documented — follow it exactly.
11. **Test with your keyboard first.** Unplug the mouse. If you can't use it, neither can keyboard users.
12. **Run NVDA + Firefox on every significant feature.** No automated tool replaces this.
13. **Bind `null` to remove ARIA attributes.** `aria-invalid="false"` adds noise. `null` removes the attribute cleanly.
14. **Don't use `aria-hidden` on focusable elements.** Focus will land on it silently. User is stranded.
15. **Test early, not after launch.** Retrofitting accessibility is 10x more expensive than building it right.

18. Resources and Further Reading

The standards

- **WCAG 2.1 full spec:** [w3.org/TR/WCAG21](https://www.w3.org/TR/WCAG21)
- **WAI-ARIA 1.2:** [w3.org/TR/wai-aria-1.2](https://www.w3.org/TR/wai-aria-1.2)
- **ARIA Authoring Practices Guide:** [w3.org/WAI/ARIA/apg/patterns](https://www.w3.org/WAI/ARIA/apg/patterns)
- **WCAG 2.2 (what's new):** [w3.org/TR/WCAG22](https://www.w3.org/TR/WCAG22)

Angular and tooling

- **Angular accessibility guide:** angular.dev/best-practices/a11y
- **Angular CDK a11y module:** material.angular.io/cdk/a11y
- **axe-core (automated WCAG testing):** github.com/dequelabs/axe-core

Free tools

- **NVDA screen reader:** nvaccess.org
- **WAVE Web Accessibility Evaluator:** wave.webaim.org
- **Colour Contrast Analyser:** tpgi.com/color-contrast-checker
- **WebAIM Screen Reader Survey:** webaim.org/projects/screenreadersurvey10

Keep learning

- **A11y Project Checklist:** a11yproject.com/checklist
- **Inclusive Design Principles:** inclusivedesignprinciples.org
- **Deque University (free resources):** dequeuniversity.com

About the Author

Sudeep Parchure is an Angular Solutions Architect focused on building accessible, performant, and inclusive web applications. This guide is based on hands-on work across Angular 21 production applications — from WCAG strategy and Angular CDK patterns through to real screen reader and keyboard testing.